Reviewer Pack — Real Radical Dimension and SOS Degree for Max-CUT

Entry 5a78c0e11f98 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 20:58:43 UTC

Real Radical Dimension and SOS Degree for Max-CUT

Entry ID: `5a78c0e11f98` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 20:58:43 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

> For a Max-CUT instance on n variables, the SOS degree required to approximate the Max-CUT value is at least Ω(log d), where d is the dimension of the real radical of the system of polynomials derived from the instance.

Rationale (proposer's reasoning):

> The real radical captures the real solutions of the polynomial system, and higher-dimensional radicals may require higher SOS degrees to approximate the Max-CUT value, as they encode more complex real constraints.

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c79b418a7eac18b9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `INCONCLUSIVE` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `real radical AND SOS degree AND Max-CUT` - `semialgebraic geometry AND sum of squares AND Max-CUT approximation` - `polynomial optimization AND real radical dimension AND SOS degree`

Top relevant hits considered: - [http://arxiv.org/abs/2512.09960v1] Numerical Behavior of the Riemann Zeta Function Using Real-to-Complex Conversion Without Gram Points or Bracketing - [http://arxiv.org/abs/physics/0401058v1] Multimer Radical Ions and Electron/Hole Localization in Polyatomic Molecular Liquids: A critical review - [http://arxiv.org/abs/1806.09998v1] Real time state monitoring and fault diagnosis system for motor based on LabVIEW - [http://arxiv.org/abs/1507.06191v1] Convexifying positive polynomials and sums of squares approximation - [http://arxiv.org/abs/2203.06291v2] Moments, Sums of Squares, and Tropicalization - [http://arxiv.org/abs/1101.4114v3] Approximating amoebas and coamoebas by sums of squares - [http://arxiv.org/abs/1710.06598v2] A Bounded Degree Lasserre Hierarchy with SOCP Relaxations for Global Polynomial Optimization and Applications - [http://arxiv.org/abs/2501.00258v1] Optimal design of frame structures with mixed categorical and continuous design variables using the Gumbel-Softmax metho

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = A[i][i]
        for j in range(i, n):
            A[i][j] /= factor
        for k in range(i+1, n):
            factor_k = A[k][i]
            for j in range(i, n):
                A[k][j] -= factor_k * A[i][j]

    # Back substitution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = A[i][-1]
        for j in range(i+1, n):
            x[i] -= A[i][j] * x[j]
    return x
```

```
def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    sign = 1
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += sign * A[0][i] * determinant(submatrix)
        sign *= -1
    return det

def real_radical_dimension(A):
    n = len(A)
    if n == 0:
        return 0
    if n == 1:
        return int(determinant(A) != 0)

    # Perform Gaussian elimination to get the rank of the matrix
    gaussian_elimination(A)
    rank = sum(1 for row in A if any(val != 0 for val in row))

    # The dimension of the real radical is n - rank
    return n - rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    A = [[random.uniform(-1, 1) for _ in range(n)] for _ in range(n)]

    d = real_radical_dimension(A)
    if d == -1:
        return {
            "metric_name": "SOS Degree",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    sos_degree = random.randint(1, 2 * n)

    return {
        "metric_name": "SOS Degree",
        "metric_value": sos_degree,
```

```
        "instances_tested": 1,
        "conjecture_holds": sos_degree >= math.log(d),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    if all(res["conjecture_holds"] for res in results):
        mean = sum(res["metric_value"] for res in results) / len(results)
        std = math.sqrt(sum((res["metric_value"] - mean)**2 for res in results) / len(results))
        support_fraction = 1.0
    else:
        mean = None
        std = None
        support_fraction = sum(res["conjecture_holds"] for res in results) / len(results)
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture

    if all(res["conjecture_holds"] or res["counterexample"] == "mapping_undefined" for res in resu
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] and res["counterexample"] != "mapping_undefined" for res
        counterexample = next(res["counterexample"] for res in results if not res["conjecture_hold
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e0272f63.py", line 111, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e0272f63.py", line 101, in run_trial
    "conjecture_holds": sos_degree >= math.log(d),
                                      ^^^^^^^^^^^
ValueError: math domain error
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with math domain error, preventing evaluation of conjecture validity | next: Fix logarithm argument validation to handle d=0/negative values

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112296
2	propose	ollama_remote	qwen3:8b	0	0	35958
3	propose	ollama_remote	qwen3:8b	0	0	111935
4	propose	ollama_remote	qwen3:8b	0	0	95305
5	preregistration	ollama_remote	qwen3:8b	0	0	24384
6	novelty	ollama_remote	qwen3:8b	0	0	20576
7	novelty	ollama_remote	qwen3:8b	0	0	24797
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16800
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12861
10	judge	ollama_remote	qwen3:8b	0	0	19691

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 474605 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5a78c0e11f98.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5a78c0e11f98.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5a78c0e11f98.tar.gz` (if generated)